

# 面向动态智能体任务的按步执行与信息共享

Yaojunzhe Zhai<sup>1</sup> · Chao-Hsien Hsieh<sup>2</sup>

<sup>1</sup> Monash University Malaysia <sup>2</sup> Xi'an International University

## 摘要

智能体的后续步骤可能改变执行位置与输入需求；请求准备完成后，源值、授权和可用能力仍可能变化。逐步裁剪难以兼顾跨步骤的累计披露，完整重跑则重复已完成的工作。本文演示一个按步执行系统：依据接收方已见信息规划当前已知的后续步骤，在发送前核验规划依赖，并保留仍有效的纯计算结果。独立接收进程记录实际载荷，界面将状态变化、失效请求与修复结果逐项对应。288 个受控场景的六方法比较中，局部修复比完整重跑减少 12.5% 的外部执行；后续规划比逐步贪心少披露 2.9% 的不同信息项，但重复传输字段增加 54.5%。另保留真实模型调用验证本地计算路径。演示使这些收益、代价和适用边界均可检查。

关键词：智能体执行；接收方视图；累计披露；局部修复；系统演示。

## 1 问题与系统目标

以合同计费任务为例：一个步骤计算金额，另一个步骤检查限额。外部服务可以接收原始数值，也可以接收本地计算的金额。只看当前步骤，发送一个派生值似乎更省；考虑后续步骤，向同一接收方复用已披露的输入可能更合适。若限额在发送前改变，旧请求应当失效，但不依赖限额的已完成计算仍可保留。

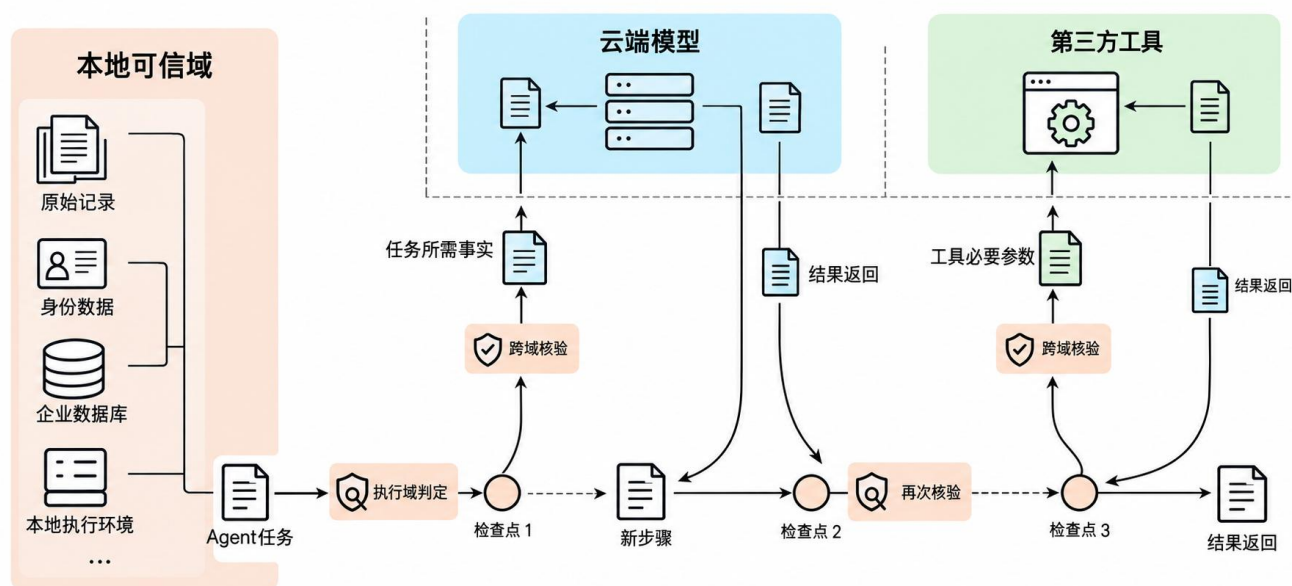


图1 按步执行的系统结构。原始资料保留在本地；模型与工具通过登记出口交互。新步骤或状态变化触发重新判定，外部结果写回本地状态。

这涉及三个相互关联的数据管理问题：计算放在哪里、每个接收方累计获知什么、状态变化后哪些结果仍有效。图1展示执行边界；本文在这一结构上实现可追溯的后续规划与局部修复。

PlanTwin [1] 研究本地规划抽象，ToolMinimize [2] 与 MINIM [3] 约束工具调用和观察视图；ATR [4] 已研究决策依赖与选择性重验证。FreshCtx [5] 提供可嵌入的上下文时效检查。因此，数据裁剪、依赖失效和程序辅助计算 [6] 本身不能作为本文的新颖性主张。

本文关注其交汇处：在信息已经发送、无法撤回的条件下，结合各接收方的披露历史，为当前已知步骤选择等价输入方案，并在变化后重建剩余执行。我们给出有限候选下的精确选择、包含未选候选的依赖核验，以及结果复用和真实接收记录的一体化实现。

演示不是将“裁剪越多越好”作为结论。观众可改变限额、撤销接收方授权或添加步骤，比较完整重跑、逐步贪心和后续规划，看到不同信息项减少时传输量仍可能上升。该取舍由可复算记录支持，而非仅由界面呈现。

## 2 面向接收方的后续执行

### 2.1 执行候选与披露历史

版本化状态  $S$  保存源事实、授权和本地能力。对登记操作  $o$ ，候选  $a$  指定接收方  $r$ 、输入视图  $V$  及其依赖。视图可以包含原值或本地派生值；登记算子须保证不同输入形式产生相同任务结果。模型不能任意增加字段或执行生成脚本。

一次披露按接收方、字段及其值与来源版本的指纹计数。令  $E(a)$  为候选披露项集合， $H$  为此前发送的集合；本地执行的  $E(a)$  为空。准备发送时保守地更新历史：

$$H_{t+1} = H_t \cup E(a_t) \quad (1)$$

同一字段发送给不同接收方分别计数；派生值也计数。源版本变化后再次发送计为新项。 $H$  不随重试或局部修复回退。该指标描述显式披露，不度量敏感性，也不排除接收方通过多个值联合推断。

### 2.2 比较当前已知的后续方案

设  $Q$  为已知待执行步骤， $P(Q, S)$  为满足当前授权与能力的候选组合。方案  $p$  的新增披露及外部执行量  $W(p)$  定义为：

$$D(p) = (\bigcup_{a \in p} E(a)) \setminus H \quad (2)$$
$$p^* = \arg \min_{p \in P(Q, S)} \text{lex}(|D(p)|, W(p)) \quad (3)$$

式（3）先减少不同披露项，再以外外部执行量打破平局。搜索逐步扩展标签  $(E, w)$ ，其中  $E$  包含历史与当前前缀的披露， $w$  为前缀外部执行数。同一深度满足下式时，可删除第二个标签：

$$E_1 \subseteq E_2 \wedge w_1 \leq w_2 \quad (4)$$

当候选输出等价、后续可行性不依赖所选表示时，为两标签追加同一后缀保持上述关系，因此剪枝不丢失式（3）的最优解。最坏标签数仍为指数级；实现超过 4,096 个标签即明确失败。这里不声称新的通用动态规划，也不声称对尚未发现的步骤全局最优。

#### 一个逐步选择失效的例子

设两个步骤均可向服务甲发送同一对原值  $\{x, y\}$ ；另一条路径向服务乙分别发送一个派生值  $\{u\}$  与两个不同派生值  $\{v, w\}$ 。逐步贪心先选乙，只披露一项；第二步无论选哪条路径，累计都至少为三项。后续规划连续选择甲，累计只有两项，但会传输四个字段。该例说明为什么披露集合与传输次数须分别衡量。

这一比较只在等价实现之间成立。若某个服务无法完成步骤，即使披露更少也不能选择；如果派生方式改变后续语义，则不能使用上述剪枝保证。登记与授权先限定可行方案，式（3）只在其中进行选择。

### 2.3 核验决策，而不只核验请求字段

请求未改变，不代表原决定仍合适。例如，本地能力变为可用时，旧外发载荷可以逐字相同。系统因此记录所有候选的源值与能力、授权、端点读取，包括未选候选；派生字段展开为传递依赖。

$$\Delta = \Delta_{\text{值}} \cup \Delta_{\text{规划}}, \forall k \in \Delta: v(k) = v_{\text{当前}}(k) \quad (5)$$

凭据将这些版本、接收方与最终序列化请求绑定。任一依赖变化即放弃未发送请求，重新规划；没有使用的源字段变化不触发失效。单调版本号能发现值变化后又恢复的情况。版本核验是保守条件，失效不一定意味着旧请求会产生错误答案。

#### 算法 1 面向接收方的后续规划与局部修复

**Input:** versioned state  $S$ , pending steps  $Q$ , registry  $R$   
**Output:** checked results  $C$  and disclosure history  $H$

```
1:  $C \leftarrow \emptyset; H \leftarrow \emptyset$ 
2: while  $Q$  is not empty do
3:    $A, \Delta \leftarrow \text{EnumerateAlternatives}(S, Q, R)$ 
4:    $p \leftarrow \text{FrontierPlan}(A, H)$ 
5:   if  $p$  is infeasible then return BLOCKED
6:    $a \leftarrow \text{First}(p); B \leftarrow \text{Serialize}(a)$ 
7:    $T \leftarrow \text{Bind}(a, \text{Versions}(S, \Delta), \text{Hash}(B))$ 
8:   if not  $\text{Current}(T, S)$  then
9:      $C \leftarrow \text{KeepValidPureResults}(C, S)$ 
10:     $Q \leftarrow \text{PendingFromCurrentState}(C, S)$ 
11:    continue
12:   if  $a.\text{recipient} = \text{local}$  then  $y \leftarrow \text{Evaluate}(a)$ 
13:   else
14:      $\text{RequireUnchanged}(T, B)$ 
15:      $H \leftarrow H \cup \text{DisclosureTokens}(a)$ 
16:      $y, \text{receipt} \leftarrow \text{SendOnce}(B)$ 
17:      $\text{VerifyReceipt}(\text{receipt}, B)$ 
18:      $C \leftarrow \text{SavePureResult}(C, y, \text{ValueDeps}(a))$ 
19:      $Q \leftarrow \text{RemoveCompletedAndAppendNew}(Q, y)$ 
20: return  $C, H$ 
```

结果缓存只适用于无副作用的登记计算，按值依赖决定是否保留；授权变化不能撤回此前披露，但会限制后续发送。没有可行候选时停止。未知副作用和不确定的网络失败不自动重试，控制器与登记出口须可信。

接收进程保存原始请求字节、摘要和运行标识，以核对“准备发送”与“实际收到”。该记录不构成远端平台认证，也不提供分布式原子提交保证。**FreshCtx** 可替换版本检查部分，后续规划与结果复用仍由本系统负责。

实现区分两类依赖：规划依赖决定是否要重新选择执行方式，值依赖决定既有结果能否复用。例如，限额变化使待执行的限额检查失效，但先前金额计算只读取基数、比例和天数，其结果仍有效。系统重新核验后保留该结果，不把“所有旧工作都作废”作为默认恢复方式。



图 2 限额变化案例的实际执行截图。① 已完成金额计算后, 限额改变; ② 旧请求失效, 系统保留不依赖限额的结果; ③ 右侧展示重新规划后的接收方与实际输入。截图裁去导航并添加编号, 使用独立开发样例, 不计入冻结测试。

### 3 交互演示

界面沿用客户端的双栏任务布局。左侧执行自动推进, 右侧随选中步骤展示接收方、发送内容、保留字段与接收记录。观众可暂停跟随检查旧请求, 再回到当前步骤; 无需逐步点击才能完成任务。展示节奏单独放慢, 不计入实验耗时。

第一组演示从计费与限额案例开始。观众先运行后续规划, 再切换完整重跑, 比较相同输入下哪些计算被重复执行。第二个案例撤销本地能力, 观察工作转至可用的外部算子; 第三个撤销某个接收方授权, 观察未发送请求失效和改路。第四个在运行中注入新的登记步骤, 显示规划只使用当时已知的信息。

这些控制场景使用独立进程执行真实 HTTP 算子, 不调用语言模型。它们隔离状态变化与修复行为, 不用于证明模型自主发现步骤。真实模型路径另提供保存的 DeepSeek 调用: 缺少事实时申请补充、调用资料查询, 以及根据表结构提出本地表达式。重放原始调用与读者自带密钥的现场调用明确区分。

界面和执行器作为独立研究实现公开, 不包含商业客户端。每轮可下载原始接收记录与完整轨迹。无密钥即可运行控制案例、核对冻结数据或重放已有模型调用; 双语界面展示相同执行状态。

### 4 实验设计

控制实验在运行前冻结源码、场景与评价规则。四种步骤顺序各取六组数值, 交叉十二类变化, 共 288 个场景、六种方法与 1,728 次执行。变化涵盖无变化、未用字段、源值、限额、本地能力增减、接收方撤权或替换、全部撤权、版本往返、新步骤和步骤间变化。参数变体不是独立真实业务样本。

六种方法为: 仅检查载荷直接字段的消融、完整重跑、局部修复与逐步贪心、局部修复与后续规划, 以及 FreshCtx 分别搭配完整重跑和后续规划。除消融外, 方法获得相同的完整依赖。FreshCtx 固定为 0.16.0, 实际调用其观察—推理—执行检查; 重跑策略由本实验提供, 不是该库的固有限制。

评价区分结果正确、版本契约符合和授权有效。每种方法有 264 个可完成场景及 24 个必须阻止的全部撤权场景; 正确阻止计为契约符合, 不计为完成任务。独立脚本重建状态、重算算子并核对全部 3,684 条 HTTP 接收记录, 同时统计不同披露项、重复传输字段、外部执行和结果复用。

另保留基于 FinQA [7] 的独立模型实验: 60 页来自 57 份公司年报, 五种方法各重复两次, 共 600 次任务、1,210 次真实 DeepSeek 调用。表结构对照已修正, 开发用公司年报被排除。该数据评价模型提出本地计算的有效性, 不与控制实验合并, 也不作为局部修复算法的效果证据。

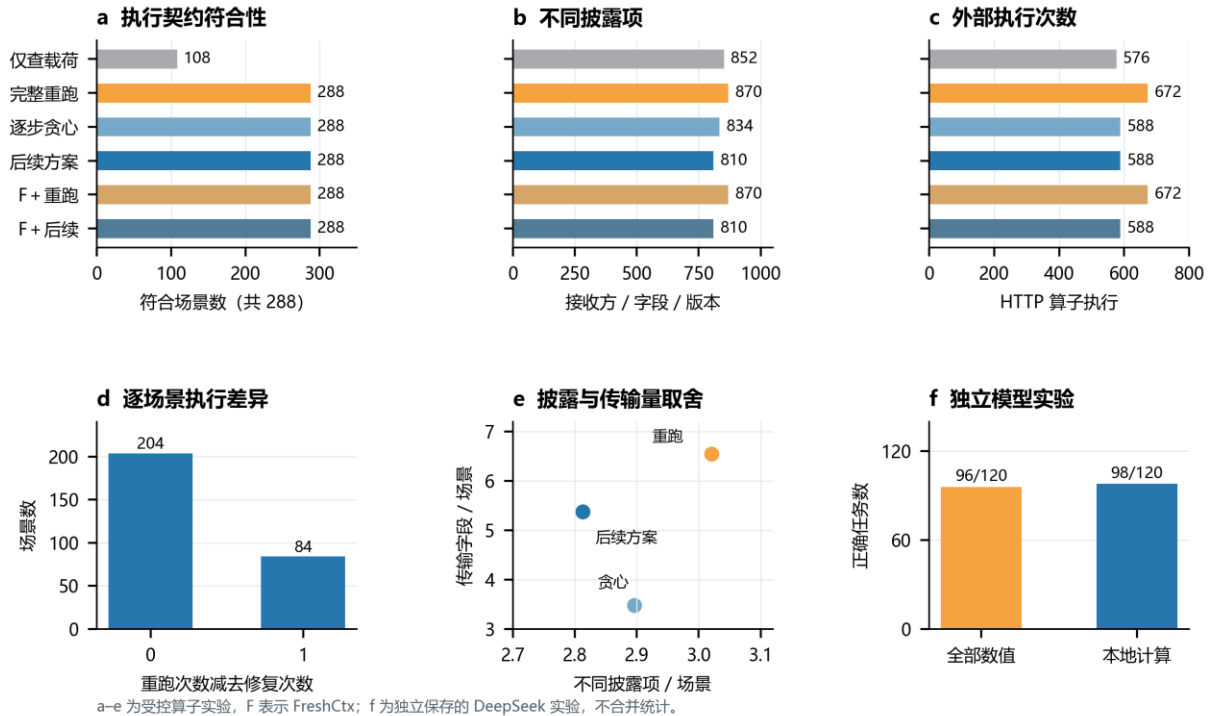


图 3 a-e：288 个受控场景的六方法比较，F 表示 FreshCtx；契约符合包含正确阻止。d：完整重跑与后续规划的逐场景执行次数差；e：不同披露项与重复传输量的取舍。f：独立保存的真实模型数值实验，不能与 a-e 合并解释。

## 5 结果与讨论

完整依赖方法均在 264 个可行场景中给出正确结果，并阻止全部 24 个撤销场景。仅查载荷的消融符合 108/288 个场景，出现 180 次过期执行，其中有 48 次未获授权。过期还包含版本往返或安全的能力变化，不能全部称为数据泄露。

后续规划的外部执行为 588 次，完整重跑为 672 次，减少 12.5%；84 个场景各少一次，204 个不变。局部修复累计复用 180 个仍有效结果。与逐步贪心相比，不同披露项从 834 降为 810，减少 2.9%；但传输字段从 1,002 增为 1,548，增加 54.5%。这是集合目标偏向重复利用已有信息的代价，不能据此声称通信更少。

FreshCtx 两个组合分别复现相应的重跑与后续规划结果，共执行 576 次外部时效检查。这说明检查层可以复用，新增行为来自披露历史、规划目标与修复策略；不能解读为本系统在时效检查上优于 FreshCtx。

独立模型实验中，全量数值通过 96/120 次，本地计算为 98/120。后者不发送业务数值，但仍公开问题、结构、年份和单位。按年报聚类的通过率差区间为  $[-5.83, 9.02]$  个百分点，不能证明准确率优势或不劣；自由文本真人盲审仍待完成。

## 6 适用范围与开放实现

本系统适用于已登记、可提供等价输入的操作。精确选择限于当前已知步骤，最坏搜索代价为指数级；自动识别语义充分视图、任意工具副作用及真实生产部署均未验证。披露项减少不等于隐私风险同比降低，接收方串通与控制旁路也不在保证范围内。

仓库提供完整源码、双语界面、固定依赖、协议、原始请求及独立核验脚本；所有失败保留。模型调用与控制实验分别存档。演示的贡献在于将执行位置、累计披露和变化后的修复连成可操作、可核对的过程，使观众能检查选择依据及其代价。

代码与复现：<https://github.com/Zane-0260907/stepwise-disclosure-demo>

## 参考文献

- [1] G. Yu 等. PlanTwin: Privacy-Preserving Planning Abstractions for Cloud-Assisted LLM Agents. arXiv:2603.18377, 2026.
- [2] W. Li, Y. Xu. ToolMinimize: Auditing and Rewriting LLM Agent Tool Calls to Minimize Privacy Exposure. arXiv:2608.24957, 2026.
- [3] H. Yu 等. MINIM: Privacy-Aware Minimal View for Agents via Trusted Local Sanitization. arXiv:2606.13949, 2026.
- [4] Y. Lyu, Y. Ren, R. Lai, W. Liu. From Version Conflicts to Decision Conflicts: Selective Revalidation for Long-Running AI Agents. arXiv:2609.08015, 2026.
- [5] Hyperwise. FreshCtx 0.16.0. [github.com/Hyperwise-LLC/freshctx](https://github.com/Hyperwise-LLC/freshctx), 2026. Apache-2.0.
- [6] L. Gao 等. PAL: Program-aided Language Models. ICML, 2023, 10764–10799.
- [7] Z. Chen 等. FinQA: A Dataset of Numerical Reasoning over Financial Data. EMNLP, 2021, 3697–3711.